CO2 ASSIGNMENT

Precedence, Short-Circuit Evaluation and Bitwise Operators in C

Name	BAVADARANI S
Register Number	1912260562

Q1 - Precedence, Associativity and the Discount Bug (6 marks)

1.1 Code under analysis

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total; /* Rs 50 off above Rs 1000 */
count = count++ + 1;
```

1.2 Rules used

- **Precedence (high to low):** * / (same level) > + - (same level) > > > ?: > =.
- **Associativity:** arithmetic operators group left-to-right; ?: and = group right-to-left.
- **Fully bracketed form of statement 1:** total = ((price*qty) - (((((price*qty)*15)/100)*senior))) + (((price*qty)*5)/100);
- Precedence and associativity fix only the **grouping**. The order in which independent operands (e.g. the three price*qty products) are evaluated is unspecified by C.

1.3 Predictions BEFORE running (all variables int)

Bill	price	qty	senior	Predicted total	Predicted net	Why
1	120	3	1	324	324	gross 360, discount 54, GST on 360 = 18; 324 is not > 1000
2	45	7	0	330	330	gross 315, discount 0, GST 315*5/100 = 15 (15.75 truncated)
3	500	3	1	1350	1300	gross 1500, discount 225, GST on 1500 = 75; 1350 > 1000 so minus 50

1.4 Operator-by-operator order (Bill 1: price=120, qty=3, senior=1)

Step	Operator applied	Computation	Result
1	* (price * qty), left product	120 * 3	360
2	* (... * 15)	360 * 15	5400
3	/ (... / 100)	5400 / 100	54
4	* (... * senior)	54 * 1	54
5	- (first term minus discount term)	360 - 54	306
6	* (price * qty) again, then * 5	360 * 5	1800
7	/ (... / 100)	1800 / 100	18
8	+ (left result + GST term)	306 + 18	324

Step	Operator applied	Computation	Result
9	= (right-to-left, assigns)	total = 324	total = 324

Steps 6 and 7 do not depend on steps 2-5, so a compiler may perform them earlier; only the dependencies in the parse tree are fixed. For the second statement: > is applied first (324 > 1000 gives 0); ?: then selects only the false branch total, so total - 50 is never evaluated; finally = stores 324 in net. For Bill 3: 1350 > 1000 is 1, so total - 50 = 1300 is evaluated and assigned.

1.5 Program and observed output

q1.c

```
#include <stdio.h>

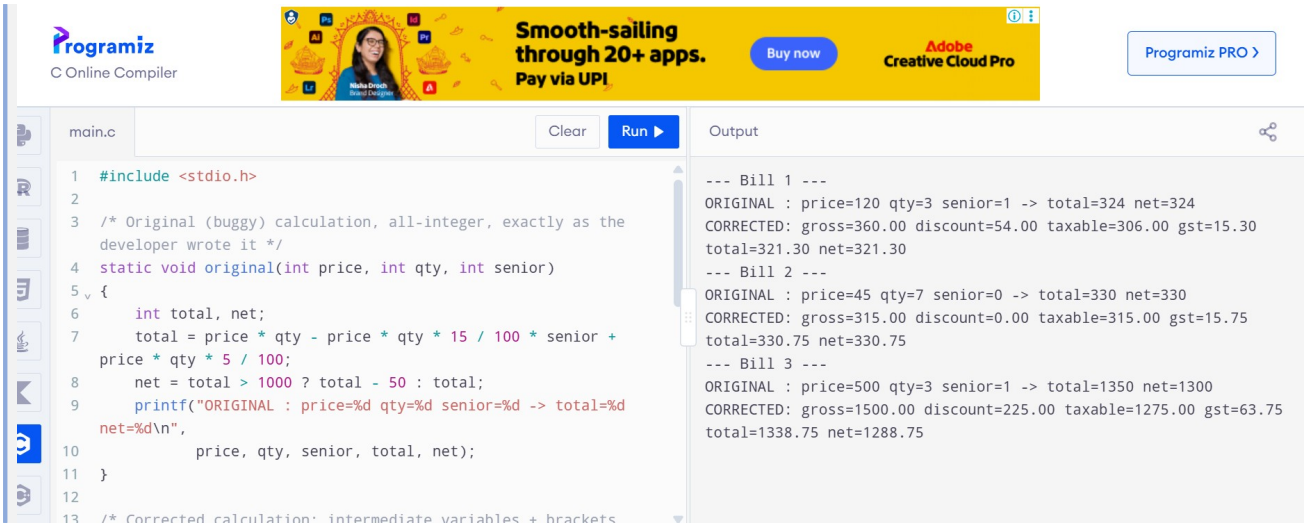
/* Original (buggy) calculation, all-integer, exactly as the developer wrote it */
static void original(int price, int qty, int senior)
{
    int total, net;
    total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    net = total > 1000 ? total - 50 : total;
    printf("ORIGINAL : price=%d qty=%d senior=%d -> total=%d net=%d\n",
           price, qty, senior, total, net);
}

/* Corrected calculation: intermediate variables + brackets, double arithmetic */
static void corrected(int price, int qty, int senior)
{
    double gross    = (double)price * qty;
    double discount = senior ? gross * 15.0 / 100.0 : 0.0;
    double taxable  = gross - discount;
    double gst      = taxable * 5.0 / 100.0;
    double total    = taxable + gst;
    double net      = (total > 1000.0) ? (total - 50.0) : total;
    printf("CORRECTED: gross=%.2f discount=%.2f taxable=%.2f gst=%.2f total=%.2f net=%.2f\n",
           gross, discount, taxable, gst, total, net);
}

int main(void)
{
    int bills[3][3] = { {120, 3, 1}, {45, 7, 0}, {500, 3, 1} };
    for (int i = 0; i < 3; i++) {
        printf("--- Bill %d ---\n", i + 1);
        original(bills[i][0], bills[i][1], bills[i][2]);
        corrected(bills[i][0], bills[i][1], bills[i][2]);
    }
    return 0;
}
```

Observed output

```
--- Bill 1 ---
ORIGINAL : price=120 qty=3 senior=1 -> total=324 net=324
CORRECTED: gross=360.00 discount=54.00 taxable=306.00 gst=15.30 total=321.30 net=321.30
--- Bill 2 ---
ORIGINAL : price=45 qty=7 senior=0 -> total=330 net=330
CORRECTED: gross=315.00 discount=0.00 taxable=315.00 gst=15.75 total=330.75 net=330.75
--- Bill 3 ---
ORIGINAL : price=500 qty=3 senior=1 -> total=1350 net=1300
CORRECTED: gross=1500.00 discount=225.00 taxable=1275.00 gst=63.75 total=1338.75 net=1288.75
```



Bill	Original total (net)	Intended total (net)	Difference	Prediction matched?
1	324 (324)	321.30 (321.30)	customer overcharged Rs 2.70	Yes
2	330 (330)	330.75 (330.75)	customer undercharged Rs 0.75	Yes
3	1350 (1300)	1338.75 (1288.75)	customer overcharged Rs 11.25	Yes

1.6 Expressions that differ from the pharmacist's intention

1. **GST computed on the wrong base (precedence/grouping bug).** The third term price*qty*5/100 taxes the gross amount, not the discounted amount. The intended rule is "(gross - discount) + 5% GST on that". Without brackets around gross - discount, the 5% is applied before the discount. This is why seniors are overcharged (Bills 1 and 3). For a non-senior (Bill 2) the grouping happens to be harmless.
2. **Integer division truncation.** price*qty*15/100 and price*qty*5/100 drop the fractional rupee: 315*5/100 = 15, not 15.75 (Bill 2 loses Rs 0.75). Since * and / are left-to-right, multiplying first and dividing last limits the loss to one truncation; writing price*qty*(15/100) would be far worse because 15/100 = 0 in integers and the discount would vanish completely.
3. * senior as a switch.** It works only if senior is exactly 0 or 1. A value such as 2 (or a character flag) would give a 30% discount. A conditional (senior ? ... : 0) states the intent.
4. **Second statement.** Its precedence is correct (> binds tighter than ?:, which binds tighter than =), but it receives the wrong, truncated total. At the boundary the error changes the outcome: a true payable of 1000.50 is stored as 1000, so the Rs 50 concession is wrongly refused.
5. **Third statement** count = count++ + 1; is undefined behaviour (see Inquiry).

1.7 Inquiry - why is `count = count++ + 1;` undefined behaviour?

In C (C11 6.5p2), if a side effect on a scalar object is unsequenced relative to another side effect on the same object, or to a value computation using that object, the behaviour is undefined. Here count is modified twice with no sequencing between the two writes: the post-increment count++ stores count+1 at some unspecified moment before the next sequence point, and the assignment = also stores a new value into count. Operators like = and + do not introduce a sequence point; the assignment only sequences its value computation (not the side effect of ++) before its own store. The language therefore allows any result (or any program behaviour), and different compilers or optimisation levels may legitimately differ. In general, an object must not be modified more than once, or modified and also read for another purpose, between two sequence points.

What the compiler does with -Wall. -Wall includes -Wsequence-point. gcc 13.3 printed:

```
q1ub.c: In function 'main':
q1ub.c:5:11: warning: operation on 'count' may be undefined [-Wsequence-point]
    5 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~~
```

It is only a warning: the program still compiles. With count = 5 it printed count = 6 at both -O0 and -O2 here, but that is an accident of this compiler, not a guarantee. A different statement such as i = i++ + 5 shows how results can vary. Using -Werror turns the warning into an error. The fix is simply count = count + 1; or count++; (one modification, no reuse).

1.8 Design decision - correct and readable rewrite

```
double gross    = (double)price * qty;                /* 1. price x quantity     */
double discount = senior ? gross * 15.0 / 100.0 : 0.0; /* 2. 15% for seniors only */
double taxable  = gross - discount;                   /* 3. amount after discount */
double gst      = taxable * 5.0 / 100.0;              /* 4. 5% GST on taxable     */
double total    = taxable + gst;                      /* 5. amount payable        */
double net      = (total > 1000.0) ? (total - 50.0) : total;  /* 6. Rs 50 off     */
count = count + 1;
```

Observed (corrected function in q1.c): Bill 1 total 321.30, Bill 2 total 330.75, Bill 3 total 1338.75 and net 1288.75 - exactly the intended values and different from the original in every bill.

Decision: use BOTH intermediate variables and brackets, but each only where it earns its place.

- Intermediate variables give each business rule (gross, discount, taxable, GST) its own named line, so the order of application is explicit, each rule can be checked or changed independently, and values can be inspected in a debugger.
- Brackets are used where precedence would otherwise not match intent, e.g. (total > 1000.0) ? (total - 50.0) : total and the (double)price * qty cast; the key grouping gross - discount is made impossible to miss by naming it taxable.
- A single bracketed expression such as total = (gross - gross*15/100*senior) * 105 / 100; would also be correct, but it hides the rules and is easy to break on the next edit.
- Money is computed in double here to show the result clearly; in real billing software the amounts should be kept in integer paise (e.g. long) with explicit rounding, to avoid both truncation and binary floating-point error.

1.9 Alternative concept and complexity analysis

Replacing the single long expression by several statements with temporaries (gross, discount, tax) gives the same arithmetic result as the corrected code above:

```
gross    = price * qty;
discount = gross * 15 / 100 * senior;   /* or the senior ? : form */
tax      = (gross - discount) * 5 / 100;
total    = gross - discount + tax;
```

Aspect	Single long expression	Intermediate variables
Operations (time)	10 arithmetic operations (price*qty computed 3 times: 1 + 4 + 3 + 2 add/sub); O(1)	9 operations (price*qty computed once); O(1). Difference is a tiny constant; an optimising compiler (common-subexpression elimination) produces nearly identical machine code for both.

Aspect	Single long expression	Intermediate variables
Extra space	None (compiler temporaries in registers)	3 extra variables (4 or 8 bytes each, tens of bytes): O(1), normally kept in registers anyway
Correctness	Easy to mis-group; the GST-on-gross bug hides in it	Each rule is a separate line, order is explicit, the bug cannot occur unnoticed, and each step is testable
Readability	Poor: one line mixing three rules	Good: self-documenting names, easy to maintain

Conclusion: the time complexity does not change (constant work per bill, O(1)), and the space cost is negligible, but correctness and readability improve substantially, so the intermediate-variable version is preferred.

Q2 - When Does Short-Circuit Evaluation Matter? (4 marks)

2.1 Experimental design

Concession rule: (senior && class_ok) || disability_card, where verify_card() is the slow server call and is the only function whose calls are counted. 1,000 sample passengers were generated deterministically: 250 seniors (25%), 334 with an eligible class, 84 who are both senior and eligible, and 50 disability-card holders. Experiment 1 uses functions that print when evaluated; Experiment 2 counts the calls to verify_card() for different orderings and operators; Experiment 3 repeats it with an audit-logging verifier.

q2.c

```
#include <stdio.h>

#define N 1000
static int calls = 0;           /* number of calls to the slow function */
static int audit_log = 0;       /* passengers recorded in the audit log */

typedef struct { int senior, class_ok, card; } Passenger;
static Passenger p[N];

/* Slow verification of the disability card (server call). */
static int verify_card(const Passenger *x) { calls++; return x->card; }
/* Changed condition: same call but also writes an audit-log record. */
static int verify_card_audit(const Passenger *x) { calls++; audit_log++; return x->card; }

static int A1(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].senior && p[i].class_ok) || verify_card(&p[i])) g++; return g; }
static int A2(void) { int g = 0; for (int i=0;i<N;i++) if (verify_card(&p[i]) || (p[i].senior && p[i].class_ok)) g++; return g; }
static int A3(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].class_ok && p[i].senior) || verify_card(&p[i])) g++; return g; }
static int A4(void) { int g = 0; for (int i=0;i<N;i++) if (((p[i].senior && p[i].class_ok) | verify_card(&p[i]))) g++; return g; }
static int A5(void) { int g = 0; for (int i=0;i<N;i++) { if (p[i].senior) { if (p[i].class_ok) { g++; continue; } } if (verify_card(&p[i])) g++; }
return g; }
static int A6(void) { int g = 0; for (int i=0;i<N;i++) if (((p[i].senior & p[i].class_ok) | verify_card(&p[i]))) g++; return g; }
static int A7(void) { int g = 0; for (int i=0;i<N;i++) { int c = verify_card_audit(&p[i]); if (c || (p[i].senior && p[i].class_ok)) g++; } return
g; }
static int A8(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].senior && p[i].class_ok) || verify_card_audit(&p[i])) g++; return g; }

static int f(int v, const char *n) { printf("   %s evaluated\n", n); return v; }

int main(void)
{
   /* Sample data: 25% senior, 33% eligible class, 5% hold a disability card */
   int s=0, e=0, c=0, both=0;
   for (int i = 0; i < N; i++) {
      p[i].senior   = (i % 4 == 0);
      p[i].class_ok = (i % 3 == 0);
      p[i].card     = (i % 20 == 7);
      s += p[i].senior; e += p[i].class_ok; c += p[i].card; both += p[i].senior && p[i].class_ok;
   }
   printf("Sample: %d passengers, %d seniors, %d eligible class, %d senior&&eligible, %d card holders\n\n", N, s, e, both, c);

   printf("Experiment 1: when is the 2nd operand skipped?\n");
   printf(" f(0,\"A\") && f(1,\"B\"):\n"); int r1 = f(0,"A") && f(1,"B"); printf("   result=%d\n", r1);
   printf(" f(1,\"A\") && f(1,\"B\"):\n"); int r2 = f(1,"A") && f(1,"B"); printf("   result=%d\n", r2);
   printf(" f(1,\"A\") || f(1,\"B\"):\n"); int r3 = f(1,"A") || f(1,"B"); printf("   result=%d\n", r3);
   printf(" f(0,\"A\") || f(1,\"B\"):\n"); int r4 = f(0,"A") || f(1,"B"); printf("   result=%d\n", r4);
   printf(" f(0,\"A\") &  f(1,\"B\"):\n"); int r5 = f(0,"A") &  f(1,"B"); printf("   result=%d\n", r5);
   printf(" f(1,\"A\") |  f(1,\"B\"):\n"); int r6 = f(1,"A") |  f(1,"B"); printf("   result=%d\n", r6);
   printf(" 2 && 1 = %d,  2 & 1 = %d\n\n", 2 && 1, 2 & 1);

   struct { const char *name; int (*fn)(void); } t[] = {
      {"V1 (senior&&class) || card()          ", A1},
      {"V2 card() || (senior&&class)          ", A2},
      {"V3 (class&&senior) || card()          ", A3},
      {"V4 nested if, card() last             ", A5},
      {"V5 (senior&&class) | card()  [bitwise]", A4},
      {"V6 (senior&class) | card()   [bitwise]", A6},
   };
   printf("Experiment 2: calls to slow verify_card() for N=%d\n", N);
```

```
    for (int k = 0; k < 6; k++) { calls = 0; int g = t[k].fn(); printf(" %s calls=%4d  concessions=%d\n", t[k].name, calls, g); }

    printf("\nChanged condition (audit log, every passenger must be recorded):\n");
    calls = 0; audit_log = 0; int g = A8(); printf(" short-circuit order V1  : calls=%d audit_records=%d concessions=%d\n", calls, audit_log,
g);
    calls = 0; audit_log = 0; g = A7();     printf(" call first, then test   : calls=%d audit_records=%d concessions=%d\n", calls, audit_log, g);
    return 0;
}
```

Observed output

```
Sample: 1000 passengers, 250 seniors, 334 eligible class, 84 senior&&eligible, 50 card holders

Experiment 1: when is the 2nd operand skipped?
 f(0,"A") && f(1,"B"):
   A evaluated
   result=0
 f(1,"A") && f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(1,"A") || f(1,"B"):
   A evaluated
   result=1
 f(0,"A") || f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(0,"A") &  f(1,"B"):
   A evaluated
   B evaluated
   result=0
 f(1,"A") |  f(1,"B"):
   A evaluated
   B evaluated
   result=1
 2 && 1 = 1,  2 & 1 = 0

Experiment 2: calls to slow verify_card() for N=1000
 V1 (senior&&class) || card()          calls= 916  concessions=134
 V2 card() || (senior&&class)          calls=1000  concessions=134
 V3 (class&&senior) || card()          calls= 916  concessions=134
 V4 nested if, card() last             calls= 916  concessions=134
 V5 (senior&&class) | card()  [bitwise] calls=1000  concessions=134
 V6 (senior&class) | card()   [bitwise] calls=1000  concessions=134

Changed condition (audit log, every passenger must be recorded):
 short-circuit order V1  : calls=916 audit_records=916 concessions=134
 call first, then test   : calls=1000 audit_records=1000 concessions=134
```

Programiz
C Online Compiler
Client wants more options? Done. Pay via UPI
Buy now
Adobe Creative Cloud Pro
Programiz PRO

main.c

```
#include <stdio.h>

#define N 1000
static int calls = 0;             /* number of calls to the slow function */
static int audit_log = 0;         /* passengers recorded in the audit log */

typedef struct { int senior, class_ok, card; } Passenger;
static Passenger p[N];

/* Slow verification of the disability card (server call). */
static int verify_card(const Passenger *x) { calls++; return x->card; }
/* Changed condition: same call but also writes an audit-log record. */
```

Clear | Run ▶

Output

```
Sample: 1000 passengers, 250 seniors, 334 eligible class, 84 senior&&eligible, 50 card holders

Experiment 1: when is the 2nd operand skipped?
 f(0,"A") && f(1,"B"):
   A evaluated
   result=0
 f(1,"A") && f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(1,"A") || f(1,"B"):
   A evaluated
   result=1
 f(0,"A") || f(1,"B"):
```

2.2 Results and explanation

When is the second operand not evaluated? For A && B, B is skipped when A is 0 (false), because the result must be 0. For A || B, B is skipped when A is non-zero (true), because the result must be 1. In the output, f(0,"A") && f(1,"B") and f(1,"A") || f(1,"B") never evaluated B. C also guarantees left-to-right evaluation with a sequence point after the first operand, so p != NULL && p->x > 0 is safe.

Version	Calls to slow function	Concessions	Calls saved
V1 (senior && class_ok) || card()	916	134	84
V2 card() || (senior && class_ok)	1000	134	0
V3 (class_ok && senior) || card()	916	134	84
V4 nested if, card() checked last	916	134	84
V5 (senior && class_ok) | card()	1000	134	0
V6 (senior & class_ok) | card()	1000	134	0

Effect of order. Putting the slow check last (V1) skips it for the 84 passengers who already qualify through senior && class_ok (916 calls); putting it first (V2) calls it for all 1,000. All versions give the same 134 concessions - only the cost differs, because the operands have no side effects.

Difference between `&` and `&&`. && is a logical operator: it short-circuits and gives 0 or 1. & is a bitwise operator: it always evaluates both operands (V5/V6 made 1,000 calls) and the order in which they are evaluated is unspecified. & also works on bit patterns, not truth values: 2 && 1 is 1 but 2 & 1 is 0 (observed), so using & on conditions that are "non-zero true" but not exactly 1 gives wrong answers.

When should the cheapest or most-likely-false condition go first? For &&, write first the condition that is cheap and most likely to be false, because a false first operand skips the rest. For ||, write first the cheap condition that is most likely true. The expected cost of A && B is cost(A) + P(A true) x cost(B), so the expensive condition should come last and be reached rarely. Here the slow card check belongs last in the ||. Within senior && class_ok, the rarer condition (senior, 25% versus 33%) goes first.

When is the order irrelevant? When both conditions are cheap, have no side effects, and neither guards the other (V1 and V3 gave identical call counts and results). But order is NOT irrelevant when the left operand protects the right one (i < n && a[i] > 0, p != NULL && p->x), because the right side must not be evaluated if the guard fails.

2.3 Changed condition - the verifier writes an audit log

Now verify_card() has a side effect, so short-circuiting changes what the program does, not only how fast it runs. Observed: with the cheap-first order V1 only 916 passengers were checked and logged, leaving **84 passengers missing from the audit log**; calling the verifier first for everybody made 1,000 calls and 1,000 audit records, with the same 134 concessions.

- If the audit must contain **every** passenger, the verifier has to run unconditionally. The recommended order reverses: call it first, store the result, then combine: int card_ok = verify_card_audit(p); if (card_ok || (senior && class_ok)) .... Avoid hiding this behind | - an explicit statement documents the intent, and | leaves the operand order unspecified.
- If the log should record only passengers who were actually verified against the server, the cheap-first order V1 is still correct (the log is consistent with the calls made), and it saves 84 server calls.

The key lesson: once an operand has side effects (logging, counters, I/O), cost-based reordering is no longer a free optimisation; correctness requirements decide the order.

2.4 Alternative concept and complexity analysis

Let n be the number of passengers and k the number for whom senior && class_ok is already true (k = 84 in the sample). Calls to the slow function:

Version	Calls (time)	Best / worst	Notes
Short-circuit (S && C) || card()	n - k	0 / n	Minimum calls; compact
Nested if statements (card checked last)	n - k	0 / n	Same behaviour and calls as &&/||, but longer code
Non-short-circuit (S & C) | card()	n (always)	n / n	Every passenger calls the server; operand order unspecified; wrong for non-0/1 values
Slow function first card() || (S && C)	n (always)	n / n	Needed only when an audit log must be complete

All versions are O(n) in the worst case and O(1) extra space; they differ in the constant factor, which matters a lot when each call is a network round-trip. Nested ifs and &&/|| are equivalent in cost (compilers compile && into the same conditional jumps), so the choice between them is about readability; the &/| version is the only one that loses the saving.

Q3 - Engineer a Device-Status Register with Bitwise Operators (6 marks)

3.1 Bit operations on one unsigned char (bit i = device i; mask = 1u << dev)

Task	Expression	Idea
Switch ON	reg | (1u << dev)	OR with the mask forces that bit to 1; other bits unchanged
Switch OFF	reg & ~(1u << dev)	~mask has 0 only at the device bit; AND clears it, keeps the rest
Toggle	reg ^ (1u << dev)	XOR flips the bit (0 to 1, 1 to 0)
Test	(reg >> dev) & 1u	Brings the bit to position 0 and isolates it, giving exactly 0 or 1
Print in binary	((reg >> bit) & 1u) for bit = 7 down to 0	Test each bit from the most significant
Count devices ON	sum of (reg >> bit) & 1u over 8 bits	Or Kernighan's method: reg &= reg - 1 clears the lowest set bit each loop; loop count = number of ones

3.2 Completed code (TODO 1 to TODO 4) and execution

q3.c

```
#include <stdio.h>

/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return (unsigned char)(reg | (1u << dev));      /* TODO 1: set bit   */
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return (unsigned char)(reg & ~(1u << dev));     /* TODO 2: clear bit */
}

unsigned char toggle(unsigned char reg, int dev)
{
    return (unsigned char)(reg ^ (1u << dev));      /* TODO 3: flip bit  */
}

int is_on(unsigned char reg, int dev)
{
    return (reg >> dev) & 1u;                       /* TODO 4: test bit  */
}

int count_on(unsigned char reg)                     /* devices that are ON */
{
    int n = 0;
    for (int bit = 0; bit < 8; bit++) n += (reg >> bit) & 1u;
    return n;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}

int main(void)
{
```

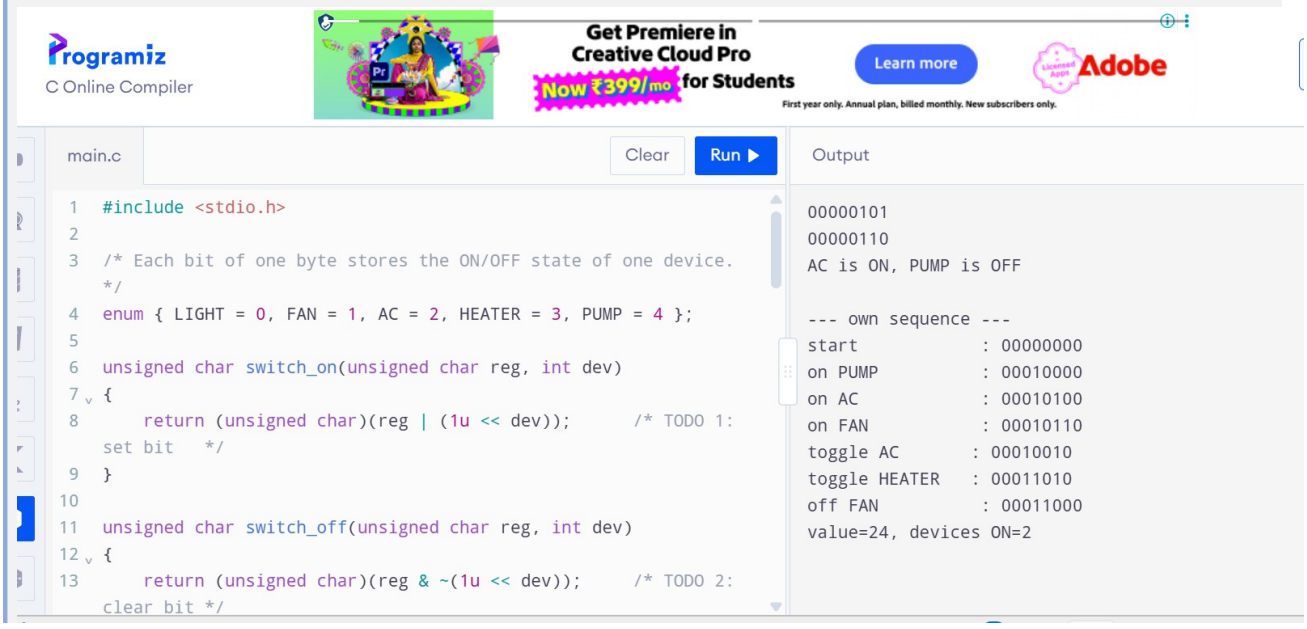
```
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");

    /* My own sequence */
    printf("\n--- own sequence ---\n");
    reg = 0;                    printf("start          : "); show(reg);
    reg = switch_on(reg, PUMP);    printf("on PUMP        : "); show(reg);
    reg = switch_on(reg, AC);      printf("on AC          : "); show(reg);
    reg = switch_on(reg, FAN);     printf("on FAN         : "); show(reg);
    reg = toggle(reg, AC);         printf("toggle AC      : "); show(reg);
    reg = toggle(reg, HEATER);     printf("toggle HEATER  : "); show(reg);
    reg = switch_off(reg, FAN);    printf("off FAN        : "); show(reg);
    printf("value=%u, devices ON=%d\n", reg, count_on(reg));
    return 0;
}
```

Observed output

```
00000101
00000110
AC is ON, PUMP is OFF

--- own sequence ---
start          : 00000000
on PUMP        : 00010000
on AC          : 00010100
on FAN         : 00010110
toggle AC      : 00010010
toggle HEATER  : 00011010
off FAN        : 00011000
value=24, devices ON=2
```



The first three lines match the expected output given in the question exactly.

3.3 Register value after each statement in main() (predicted before running, then verified)

Statement	Decimal	Binary (predicted)	Printed?	Observed
unsigned char reg = 0;	0	00000000	-	matches

Statement	Decimal	Binary (predicted)	Printed?	Observed
reg = switch_on(reg, LIGHT);	1	00000001	-	matches
reg = switch_on(reg, AC);	5	00000101	show()	00000101
reg = toggle(reg, FAN);	7	00000111	-	matches
reg = switch_off(reg, LIGHT);	6	00000110	show()	00000110
is_on(reg, AC) / is_on(reg, PUMP)	6	bit 2 = 1, bit 4 = 0	printf	AC ON, PUMP OFF

My own sequence (predicted first, then verified by the program):

Operation	Decimal	Binary	Devices ON
start	0	00000000	none
switch_on PUMP (bit 4)	16	00010000	PUMP
switch_on AC (bit 2)	20	00010100	AC, PUMP
switch_on FAN (bit 1)	22	00010110	FAN, AC, PUMP
toggle AC	18	00010010	FAN, PUMP
toggle HEATER (bit 3)	26	00011010	FAN, HEATER, PUMP
switch_off FAN	24	00011000	HEATER, PUMP (count = 2)

Observed output for this sequence (see above) matched every prediction, ending with value = 24 and count_on = 2.

3.4 Inquiry

Why `reg & 1 << dev` and `(reg & 1) << dev` differ. The shift operators have higher precedence than &, so reg & 1 << dev means reg & (1 << dev): it masks the bit of device dev - a correct test, but the result is the mask value (1 << dev, e.g. 4 for AC), not 1, so a comparison such as == 1 fails for every device except dev 0. In (reg & 1) << dev the brackets force the AND first: it examines only bit 0 (the LIGHT), and then shifts that single bit to position dev. It tells you the state of device 0, whichever device you asked about. Observed with dev = 2:

```
reg=0x04 dev=2:  reg & 1 << dev = 4   (reg & 1) << dev = 0
reg=0x05 dev=2:  reg & 1 << dev = 4   (reg & 1) << dev = 4
reg=0x01 dev=2:  reg & 1 << dev = 0   (reg & 1) << dev = 4
```

- reg = 0x04 (AC on, LIGHT off): reg & 1 << dev = 4 (correct, AC is on) but (reg & 1) << dev = 0 (wrongly says AC is off).
- reg = 0x01 (only LIGHT on): reg & 1 << dev = 0 (correct) but (reg & 1) << dev = 4 (wrongly says AC is on).
- reg = 0x05: both give 4 only by coincidence.

What goes wrong with a signed char when bit 7 is set. Bit 7 becomes the sign bit. 0x80 does not fit in a signed char; the conversion is implementation-defined and normally gives -128. In expressions the value is promoted to int with sign extension, so it becomes 0xFFFFFF80. Observed:

```
unsigned: value=128  hex=80  >>1 = 40  ==0x80? 1
signed  : value=-128 hex=ffffff80  >>1 = fffffc0  ==0x80? 0
signed 0x80, while(t) t>>=1 : still non-zero after 20 shifts (t=-1)
unsigned 0x80 same loop      : ends after 8 shifts
```

- Printing shows -128 (and ffffff80 in hex) instead of 128 (80); reg == 0x80 is false.
- Right shift of a negative value is implementation-defined and usually arithmetic (sign-filling): reg >> 1 gives 0xFFFFFFC0 instead of 0x40, which corrupts masks and tests.
- A loop like while (reg) reg >>= 1; (e.g. for counting bits) never ends for a negative value, because it stays -1; the unsigned version ends after 8 shifts.
- Left shifting a negative value (reg << 1) is undefined behaviour in C.

The show() function appears to work with a signed char only because of the & 1u mask. The register should always be unsigned char (or uint8_t) so that all 8 bits are plain data.

3.5 Alternative concept and complexity analysis - `char state[8]`

With one array element per device the operations become state[dev] = 1; (on), state[dev] = 0; (off), state[dev] = ! state[dev]; or state[dev] ^= 1; (toggle), and state[dev] (test). No masks or shifts are needed.

Aspect	Bit-packed unsigned char	Array char state[8]
Set / clear / toggle / test one device	O(1): a shift plus one bitwise operation	O(1): one indexed memory access
Count devices ON	Loop over 8 bits, O(n) with n = 8 devices (a constant); Kernighan's method needs one pass per ON device; a 256-entry lookup table or hardware popcount makes it O(1)	Loop over 8 elements adding them, O(n) with n = 8 devices (a constant); no shortcut
Space per home	1 byte	8 bytes
Space for 1,000,000 homes	1,000,000 bytes (about 0.95 MiB)	8,000,000 bytes (about 7.63 MiB) - 8 times more
Other points	Whole state can be copied, compared (reg1 == reg2), stored or sent as one byte; many devices can be changed at once with a mask; code is harder to read and bugs like the precedence/signedness errors above are possible	Very readable and simple to debug; elements can store more than ON/OFF (e.g. fan speed); wastes memory, which matters on a low-cost controller

Conclusion: both designs take O(1) time for single-device operations and O(number of devices) for counting, so time does not decide the choice. Space does: for a million homes the bit-packed register saves about 7 MB and is the right design for a memory-limited controller, provided an unsigned type and careful bracketing of masks are used.